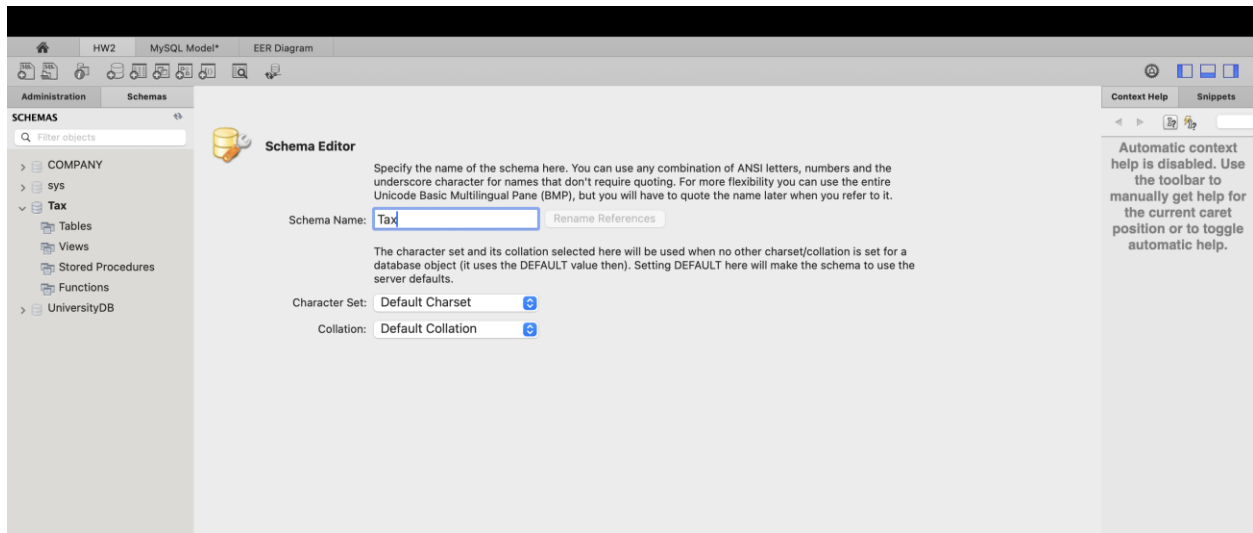


Jeevan Kumar Banoth – 02105145

CIS 552: Database Design
Homework – 4:

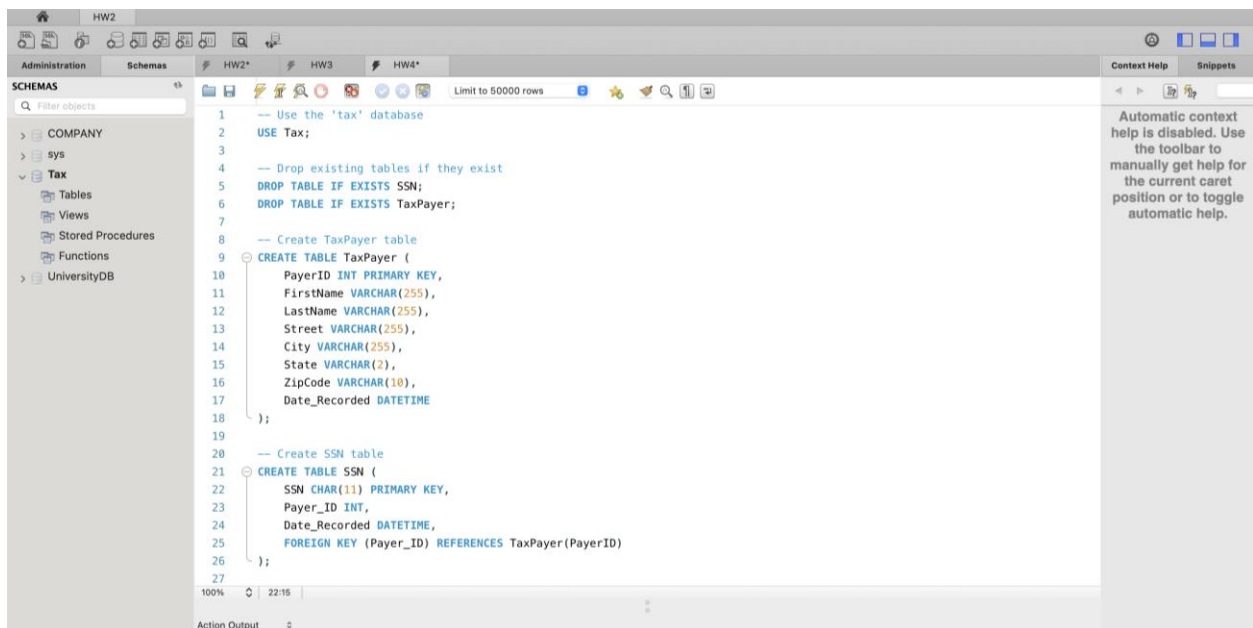
Enhancing Index Utilization for Query Efficiency.

1. Used MySQL Workbench to create a new database with the name "tax".



As you can see above, by using the schema editor I have created a Database called “Tax”.

2. Following the instructions in the supplied course diagram, we create tables, define columns, and impose constraints to establish the database's structure.

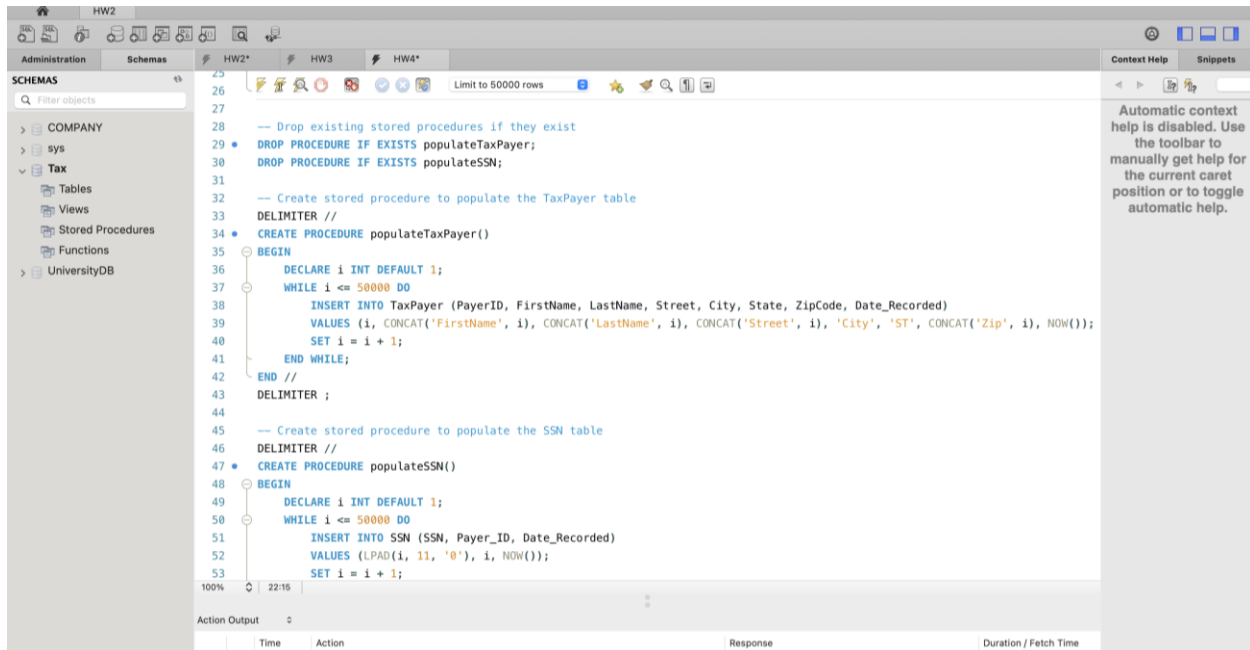


Numerous fields, including Payer ID, First and Last Names, Address Details, Zip Code, and Date Posted for every record, are included in the Taxpayer Table. Every record in the table has a unique identification, which is the payer identifier.

Under the SSN table, we have SSN CHAR (11), Player_ID INT, Date Recorded, Foreign Key etc.

Three columns make up the SSN table's design: Date_Recorded, Payer_ID, and SSN. As the primary key, the SSN column defines the CHAR(11) data type. Referential integrity between the two tables is guaranteed by the foreign key constraint on the Payer_ID column, which references the PayerID column of the TaxPayer table.

3. Using the insert scripts included in the course materials, test records are added to the Taxpayer and related tables.



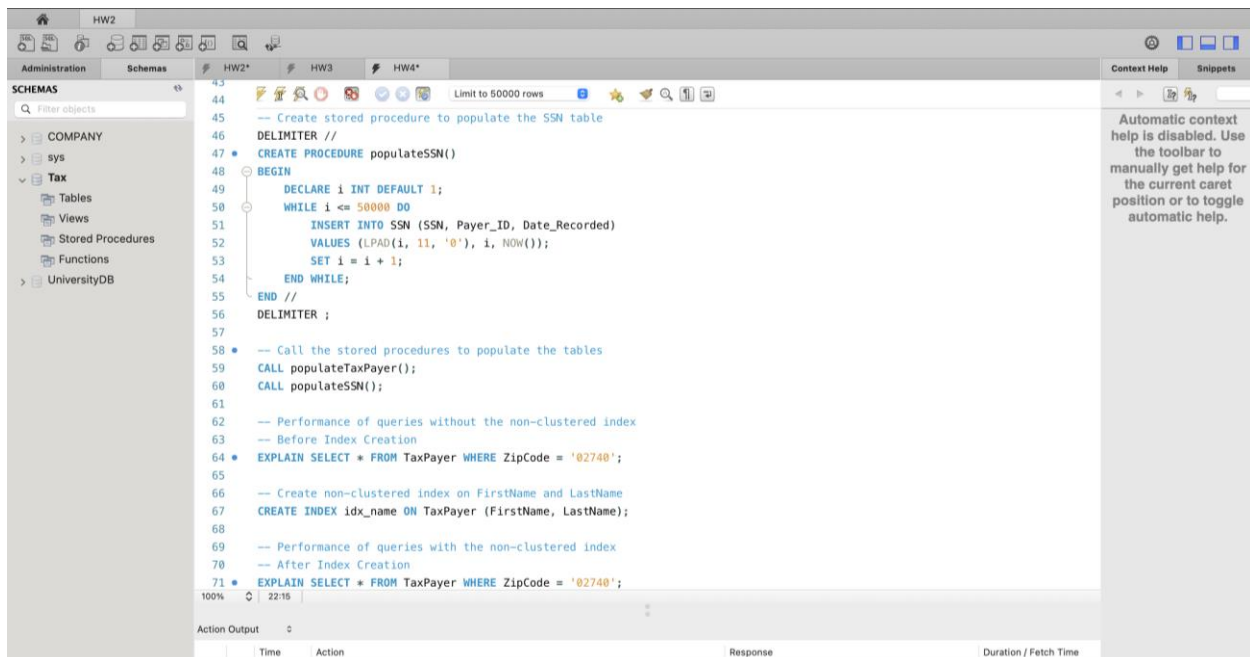
The screenshot shows a MySQL IDE interface with a SQL editor window. The editor contains the following SQL code:

```
25
26
27
28 -- Drop existing stored procedures if they exist
29 DROP PROCEDURE IF EXISTS populateTaxPayer;
30 DROP PROCEDURE IF EXISTS populateSSN;
31
32 -- Create stored procedure to populate the TaxPayer table
33 DELIMITER //
34 CREATE PROCEDURE populateTaxPayer()
35 BEGIN
36     DECLARE i INT DEFAULT 1;
37     WHILE i <= 50000 DO
38         INSERT INTO TaxPayer (PayerID, FirstName, LastName, Street, City, State, ZipCode, Date_Recorded)
39         VALUES (i, CONCAT('FirstName', i), CONCAT('LastName', i), CONCAT('Street', i), 'City', 'ST', CONCAT('Zip', i), NOW());
40         SET i = i + 1;
41     END WHILE;
42 END //
43 DELIMITER ;
44
45 -- Create stored procedure to populate the SSN table
46 DELIMITER //
47 CREATE PROCEDURE populateSSN()
48 BEGIN
49     DECLARE i INT DEFAULT 1;
50     WHILE i <= 50000 DO
51         INSERT INTO SSN (SSN, Payer_ID, Date_Recorded)
52         VALUES (LPAD(i, 11, '0'), i, NOW());
53         SET i = i + 1;
54     END WHILE;
55 END //
56 DELIMITER ;
```

The interface includes a left sidebar with a 'SCHEMAS' tree showing a database named 'UniversityDB' with tables 'COMPANY', 'sys', and 'Tax'. The main editor window has a toolbar with icons for saving, undo, redo, and other standard IDE functions. A right sidebar shows a 'Context Help' panel with a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

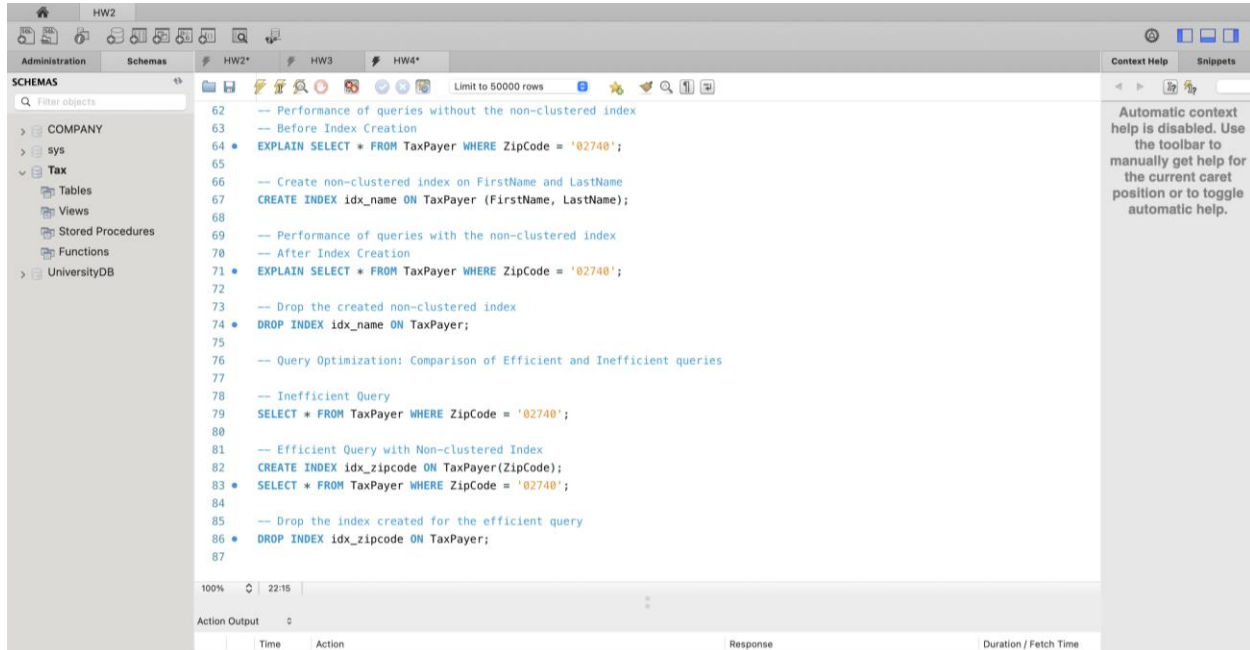
Delete the Created Non-clustered Index: Using the drop procedure.

This stored procedure approach will help in populating the tables efficiently in MySQL.



Compare Execution Plans of Efficient and Inefficient Queries: Use the EXPLAIN command to compare the execution plans and times for the efficient and inefficient queries.

4. Creating a plan of execution and evaluating the effectiveness of database queries with and without an index.



If there is no index, the database engine will have to go through each row in the table one by one to find the ones that have the zip code "02740". When the table gets bigger, it can take longer to complete this procedure since you have to look at each row separately to find entries that match.

The database engine can quickly locate all entries that match a specific zip code by using the directory as an index. An index is typically a better option than a full table scan since it allows the database to access the rows that fit the search criteria directly, without having to go through the laborious process of looking through every row.

All records in the Taxpayer table with the Zip Code "02740" are retrieved by the query. Understanding the query's performance characteristics requires knowing how MySQL would run the query, which is obtained with the EXPLAIN statement.

Executing the queries that are efficient and inefficient.

The screenshot shows a database management tool interface with a query editor and an execution plan. The query is:

```
1 -- Use the 'tax' database
2 USE Tax;
```

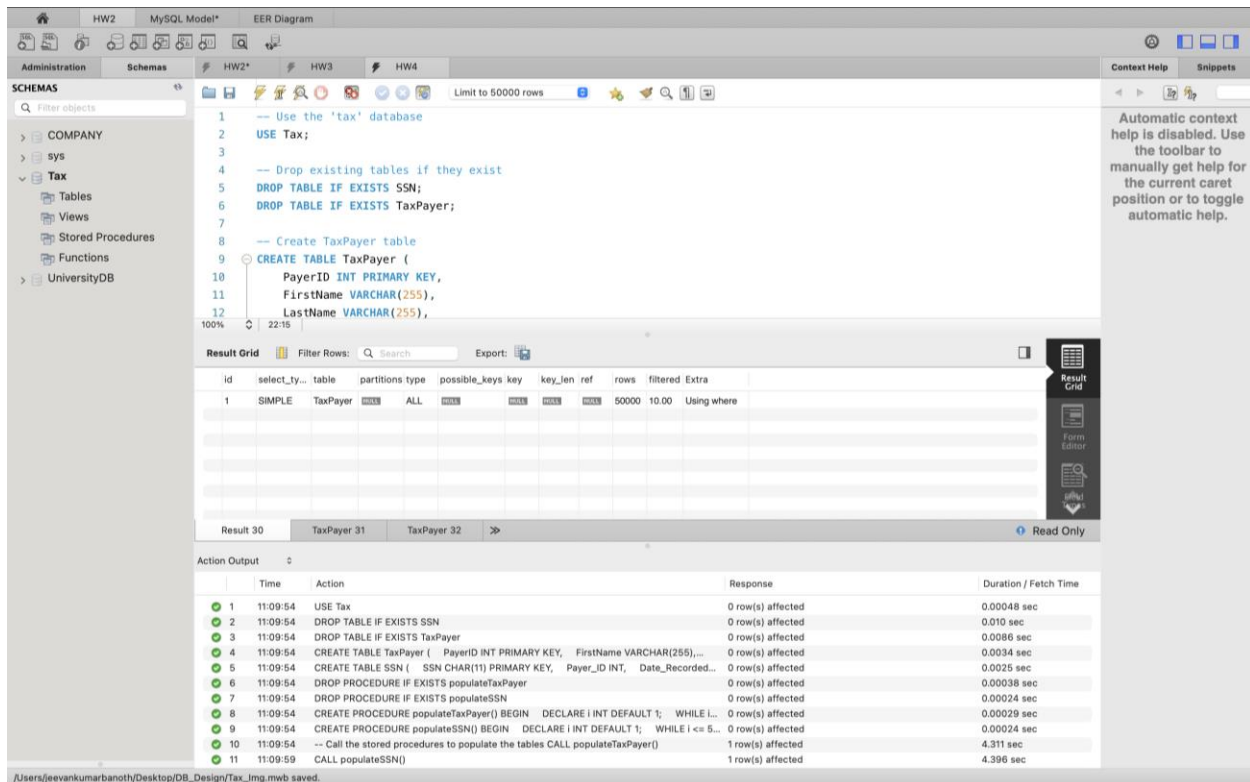
The execution plan (Action Output) shows the following steps:

	Time	Action	Response	Duration / Fetch Time
1	11:09:54	USE Tax	0 row(s) affected	0.00048 sec
2	11:09:54	DROP TABLE IF EXISTS SSN	0 row(s) affected	0.010 sec
3	11:09:54	DROP TABLE IF EXISTS TaxPayer	0 row(s) affected	0.0086 sec
4	11:09:54	CREATE TABLE TaxPayer (PayerID INT PRIMARY KEY, FirstName VARCHAR(255),...	0 row(s) affected	0.0034 sec
5	11:09:54	CREATE TABLE SSN (SSN CHAR(11) PRIMARY KEY, Payer_ID INT, Date_Recorded...	0 row(s) affected	0.0025 sec
6	11:09:54	DROP PROCEDURE IF EXISTS populateTaxPayer	0 row(s) affected	0.00038 sec
7	11:09:54	DROP PROCEDURE IF EXISTS populateSSN	0 row(s) affected	0.00024 sec
8	11:09:54	CREATE PROCEDURE populateTaxPayer() BEGIN DECLARE i INT DEFAULT 1; WHILE i <= 5...	0 row(s) affected	0.00029 sec
9	11:09:54	CREATE PROCEDURE populateSSN() BEGIN DECLARE i INT DEFAULT 1; WHILE i <= 5...	0 row(s) affected	0.00024 sec
10	11:09:54	-- Call the stored procedures to populate the tables CALL populateTaxPayer()	1 row(s) affected	4.311 sec
11	11:09:59	CALL populateSSN()	1 row(s) affected	4.396 sec
12	11:10:03	EXPLAIN SELECT * FROM TaxPayer WHERE ZipCode = '02740'	1 row(s) returned	0.00031 sec / 0.0000...
13	11:10:03	CREATE INDEX idx_name ON TaxPayer (FirstName, LastName)	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.061 sec
14	11:10:03	EXPLAIN SELECT * FROM TaxPayer WHERE ZipCode = '02740'	1 row(s) returned	0.00024 sec / 0.0000...
15	11:10:03	DROP INDEX idx_name ON TaxPayer	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.0020 sec
16	11:10:03	SELECT * FROM TaxPayer WHERE ZipCode = '02740' LIMIT 0, 50000	0 row(s) returned	0.012 sec / 0.000008...
17	11:10:03	CREATE INDEX idx_zipcode ON TaxPayer (ZipCode)	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.041 sec
18	11:10:03	SELECT * FROM TaxPayer WHERE ZipCode = '02740' LIMIT 0, 50000	0 row(s) returned	0.00022 sec / 0.0000...
19	11:10:03	DROP INDEX idx_zipcode ON TaxPayer	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.0020 sec

There is a system in place in the database's taxpayer's table to quickly find records by zip code.

The taxpayer table's lookup function goes through each record one by one, looking for ones that correspond to the given zip code. This method is typically less effective and more time-consuming without an index.

To enhance the efficiency of search operations that utilize the Taxpayers table's ZIP code field as a filter, we built a non-clustered index on it. By streamlining the database engine's search for all records with a specific zip code, this index significantly lowers the amount of time and processing power needed for query operations.



By reducing the ranks, carefully crafted queries using indexes can reduce the amount of work on the database, allowing for quicker query execution and less resource consumption. Metrics measuring query performance before and after index deployment can be compared to show how beneficial indexes are for database structures and how important they are for query performance optimization.

Conclusion:

The performance of database queries can be enhanced by employing indexes and building structured tables.

We first created the "taxes" database in MySQL Workbench and set up the tables and constraints in accordance with the schema's specifications.

Performance testing is enabled by entering name data. Criteria searches get slower as a table gets larger, and non-clustered indexes have a significant impact on query performance.

By lowering search time and resource consumption, indexes on important fields, like zip code, streamline the search process.

Index performance with and without them is compared, demonstrating the value of indexes in lowering the number of rows scanned, which enhances performance and emphasizes their crucial role in database architecture and query optimization.

Practical understanding of database management, SQL syntax customisation, and typical SQL error troubleshooting were imparted during this process. This highlights how crucial it is to pay close attention to details when creating a database schema and how subtle it might be to work with various SQL dialects.

Created non-clustered indexes on FirstName, LastName, and Zip Code to enhance query performance. Demonstrated the performance difference before and after index creation using EXPLAIN statements.

-----X-----